

Library System Example

A Semantic Foundation for Deontic Action Logic

Introduction

This document presents a comprehensive example of how deontic action logic can be applied to a library information system. The example illustrates the key concepts of possible worlds, admissibility, obligation, permission, and action transitions in a practical context that demonstrates how normative constraints shape system behaviour.

Rather than beginning with axioms or proof systems, this approach introduces semantic primitives directly in first-order logic, showing how normative concepts arise as constraints over structures. This framework naturally supports reasoning about system behaviour and can be understood by developers, audited by stakeholders, and implemented by machines.

Domain Predicates

The library system is modeled using the following predicates over possible worlds:

Predicate	Meaning
$\text{Borrowed}(b, w)$	Book b is borrowed in world w
$\text{Overdue}(b, w)$	Book b is overdue in world w
$\text{LoanedTo}(b, u, w)$	Book b is loaned to user u in world w
$\text{Reserved}(b, u, w)$	Book b is reserved by user u in world w
$\text{Due}(b, u, d, w)$	Borrowed book b by user u is due in d days in world w

System Norms

The library system is governed by several normative constraints that define which worlds are admissible:

Norm 1: Overdue Books Must Not Be Loaned

$$\neg \exists b \exists u (\text{Overdue}(b, w) \wedge \text{LoanedTo}(b, u, w))$$

This forbids any world in which an overdue book is simultaneously loaned to someone. In first-order logic:

$$\forall w (A(w) \rightarrow \neg \exists b \exists u (\text{Overdue}(b, w) \wedge \text{LoanedTo}(b, u, w)))$$

Norm 2: Reserved Books Cannot Be Borrowed by Others

$$\neg (\exists b \exists u_1 \exists u_2 (\text{Reserved}(b, u_1, w) \wedge \text{LoanedTo}(b, u_2, w) \wedge u_1 \neq u_2))$$

If a book is reserved by one user, it cannot be loaned to a different user.

Norm 3: Loan Duration Constraints

Different types of books and users have different maximum loan periods:

- Standard textbooks: 30-day loan period
- Reference books: cannot be loaned
- Rare manuscripts: 3-day loan for standard users, 5-day loan for VIP users

Admissible Worlds

A world w is admissible if it satisfies all the system norms. The admissibility predicate combines all constraints:

$$\begin{aligned} A(w) \equiv & \forall b, u, d (\text{Borrowed}(b, u, w) \rightarrow d \leq \\ & \text{AllowedBorrowDuration}(b, u, d)) \wedge \\ & \neg \exists b, u (\text{Overdue}(b, w) \wedge \text{LoanedTo}(b, u, w)) \wedge \\ & \neg \exists b, u_1, u_2 (\text{Reserved}(b, u_1, w) \wedge \text{LoanedTo}(b, u_2, w) \wedge u_1 \neq u_2) \end{aligned}$$

Example Worlds

Consider a simple scenario with two books (b_1 : Textbook, b_2 : Rare manuscript) and two users (u_1 : Standard user, u_2 : VIP user). Here are some admissible worlds:

World	State Description
w_0	Empty library: no books borrowed, no reservations
w_1	$\text{Borrowed}(b_1, u_1, w_1), \text{Due}(b_1, u_1, 30, w_1)$ — Standard user borrows textbook
w_2	$\text{Borrowed}(b_1, u_1, w_2), \text{Due}(b_1, u_1, 30, w_2), \text{Borrowed}(b_2, u_2, w_2), \text{Due}(b_2, u_2, 5, w_2)$ — Both books borrowed (VIP gets 5-day period for rare book)
w_3	$\text{Borrowed}(b_1, u_1, w_3), \text{Due}(b_1, u_1, 30, w_3), \text{Reserved}(b_2, u_1, w_3)$ — Textbook borrowed, rare book reserved by same user
w_4	$\text{Borrowed}(b_2, u_2, w_4), \text{Due}(b_2, u_2, 5, w_4)$ — VIP borrows rare book alone

Actions and Transitions

Actions represent transitions between worlds. Each action has preconditions that must be satisfied for it to be executable, and effects that define the resulting world state.

Borrow Action

Preconditions:

- Book is not currently borrowed
- Book is not reserved by a different user
- Book is not a reference book

Effects:

- Borrowed(b,u,w') becomes true
- Due date is set according to book type and user status

Return Action

Preconditions:

- Book is currently borrowed by the user

Effects:

- Borrowed(b,u,w') becomes false
- Due date information is cleared

Reserve Action

Preconditions:

- Book is not currently reserved by anyone

Effects:

- Reserved(b,u,w') becomes true

State Transition Diagram

The following diagram illustrates valid state transitions in a simplified library system. Each node represents an admissible world, and edges represent actions that transition between worlds:

Initial State (w_4):

- No books borrowed, no reservations

Possible Transitions:

- $w_4 \rightarrow w_1$: User u_1 borrows book b_1
- $w_4 \rightarrow w_3$: User u_1 reserves book b_2
- $w_1 \rightarrow w_2$: User u_2 borrows book b_2 (VIP exception allows this)
- $w_1 \rightarrow w_4$: User u_1 returns book b_1
- $w_2 \rightarrow w_1$: User u_2 returns book b_2
- $w_2 \rightarrow w_4$: Both users return their books
- $w_3 \rightarrow w_1$: User u_1 cancels reservation on book b_2

Application to Logic Auditing

This semantic model serves several important purposes in a Logic Auditor framework:

1. Early Semantic Validation

Before generating code, the Logic Auditor can explore candidate worlds to ensure they match stakeholder expectations. Any proposed action that would lead to a non-admissible world is immediately flagged.

2. Unified Reasoning About Exceptions

Exception handling (like VIP user privileges or special book types) is not implemented as ad-hoc code branches. Instead, exceptions are treated uniformly as constraints on admissibility. A world that violates any rule—even a rare exception—is automatically excluded from the set of admissible worlds.

3. Simplified Implementation

Code (whether human-written or LLM-generated) becomes a projection of admissible worlds. Exception handling emerges as a byproduct of adherence to admissibility rather than requiring bespoke logic scattered throughout the codebase.

4. Countermodel Detection

Any candidate world outside the set of admissible worlds is a countermodel. These countermodels indicate either:

- A problem in the specification of rules
- A bug in generated code
- An inconsistency in stakeholder requirements

Practical Benefits

This approach to system specification offers several concrete advantages:

Explicit Normative Commitments

All system rules are captured formally as predicates over worlds. This makes implicit assumptions explicit and allows stakeholders to verify that the formalisation matches their intent.

Machine-Checkable Specifications

The semantic model can be directly translated into model checking tools like Alloy or TLA+. This enables automated verification that an implementation satisfies the specified norms.

LLM-Assisted Code Generation

Large language models can propose candidate implementations based on the formal specification. The Logic Auditor then verifies that all proposed transitions lead to admissible worlds, catching errors before deployment.

Traceability

Every system behavior can be traced back to specific norms in the specification. This supports auditing, compliance verification, and debugging when unexpected behaviours occur.

Connection to Formal Logic

While this framework uses the intuitive language of 'worlds' and 'admissibility,' it has rigorous logical foundations:

First-Order Reduction

All deontic notions (obligation, prohibition, permission) can be reduced to first-order logic:

- $O(\varphi) \equiv \forall w \ (A(w) \rightarrow \varphi(w))$
- $F(\varphi) \equiv \forall w \ (A(w) \rightarrow \neg\varphi(w))$
- $P(\varphi) \equiv \exists w \ (A(w) \wedge \varphi(w))$

This means that deontic reasoning becomes quantifier evaluation over a first-order domain, making it amenable to automated reasoning tools.

Semantic, Not Axiomatic

Unlike traditional deontic logic systems that start with axioms (like ' $O(\varphi) \rightarrow P(\varphi)$ '), this framework defines everything semantically through the structure of admissible worlds. This avoids paradoxes and unintended commitments that arise from axiomatic approaches.

Conclusion

The library system example demonstrates how deontic action logic can provide a rigorous semantic foundation for software specification. By explicitly modeling:

- Possible system states (worlds)
- Normative constraints (admissibility)
- State properties (predicates)
- State transitions (actions)

we create a framework that bridges the gap between stakeholder requirements, formal specification, and executable code. This approach makes implicit assumptions explicit, enables automated verification, and supports principled reasoning about system behaviour.

The framework is intentionally minimal—it provides just enough structure to capture normative constraints without imposing unnecessary logical machinery. This makes it practical for real-world software development while maintaining mathematical rigour.